

TP Framebuffer

Malek.Bengougam@gmail.com

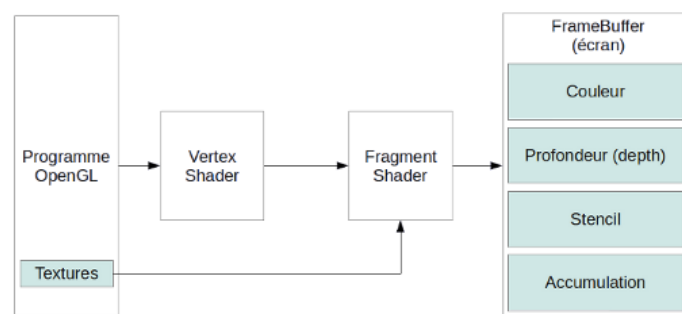
Partie 1 – Framebuffer Object

Après avoir vu les VBO (Vertex Buffer Objects), les VAO (Vertex Array Objects), les Texture Objects nous allons maintenant aborder les Framebuffer Objects (FBO).

Comme leur nom le laisse supposer, les FBO ont pour objectifs de s'utiliser comme un framebuffer c'est-à-dire comme une classe de gestion de la mémoire vidéo stockant des pixels.

Jusqu'à présent nous nous sommes reposés sur GLFW ainsi que sur l'OS pour gérer ce fameux "canvas" de rendu.

Nous avons également vu que le framebuffer (le canvas de l'écran) se compose en fait de plusieurs buffers, essentiellement un color buffer (un autre au niveau de l'OS pour le "double buffering") et un depth buffer pour le tri des faces en 3D. Optionnellement, d'autres buffers sont disponibles comme le –stencil-buffer, utile pour des effets de pochoirs ou de masquage/identification ou l'accumulation-buffer historiquement utilisé pour le super-sampling ou le motion-blur



Les FBO nous permettent de créer un ou plusieurs framebuffers supplémentaires dans le but d'effectuer des rendus sans pour autant affecté le framebuffer principal utilisé pour l'affichage dans la fenêtre.

Cette technique est souvent appelée "rendu hors-écran (offscreen rendering)".

L'usage du rendu hors-écran apporte plus de souplesse dans le pipeline de rendu afin d'implémenter des techniques telles que le post-processing, le rendu dans une texture (render-to-texture) etc...

Un FBO est essentiellement une structure qui va manager les buffers (color, depth etc...) que nous allons lui attacher, afin que le GPU puisse correctement dessiner dans les dits buffers.

Ces buffers peuvent être de deux types: des textures ou des renderbuffers. La différence tient au fait que les renderbuffers sont optimisés et préférables lorsque la zone de rendu est temporaire et n'est pas destinée à être utilisée par la suite (en écriture seule donc).

Pour ne pas apporter de la confusion entre FBO, buffers, renderbuffers, nous allons plutôt utiliser des textures ici pour ce qu'OpenGL nomme un "attachment" (point d'attache) d'un FBO.

1. Gestion d'un FBO

Un FBO se gère comme n'importe quel autre objet OpenGL à savoir à l'aide d'un entier (GLuint) pour stocker l'identifiant de l'objet. La valeur zéro (0) identifiant le FBO par défaut, qui est en fait le back-buffer créé par GLFW.

Pour créer, et respectivement détruire, un FBO on utilise les fonctions suivantes:

```
void glGenFramebuffers(int count, GLuint* objects);  
void glDeleteFramebuffers(int count, GLuint* objects);
```

Un FBO est rendu actif à travers un appel à la fonction `glBindFramebuffer()`, décrite ci-dessous. Un seul FBO peut être actif à la fois –le rendu du GPU est alors redirigé vers le FBO courant.

```
void glBindFramebuffer(GLenum type, GLuint object);
```

Les paramètres de la fonctions sont le type d'opération sur un Framebuffer pour lequel le FBO défini dans le second paramètre sera rendu actif.

Il existe 3 (trois) types d'opération:

- Lecture/Ecriture - on spécifie alors comme type **GL_FRAMEBUFFER**
- Lecture seule – on spécifie alors le type **GL_READ_FRAMEBUFFER**
- En écriture seule – on spécifie le type **GL_DRAW_FRAMEBUFFER**

Exemple 1:

```
GLuint FBO;  
glGenFramebuffers(1, &FBO);  
...  
glBindFramebuffer(GL_FRAMEBUFFER, FBO); // en lecture et écriture  
...  
glBindFramebuffer(GL_FRAMEBUFFER, 0); // rétabli le FBO par défaut, créé par l'API de  
fenêtrage  
...  
glDeleteFramebuffers(1, &FBO);
```

2. Opérations de rendu

Une fois rendu actif par un appel à `glBindframebuffer()`, un FBO est impacté par l'ensemble des fonctions d'OpenGL dédiées au rendu de fragments / pixels.

Concrètement, cela inclus les fonctions de fenêtrage comme *glViewport()* et *glScissor()* et toutes les fonctions types *glDrawArrays()*, *glDrawElements()*, *glColorMask()*, *glDepthMask()* etc...

OpenGL ayant un fonctionnement type "machine à état" cela implique que lorsque l'on change de Framebuffer courant, OpenGL conserve sa configuration.

Supposons un cas d'école où nous souhaitons effectuer un rendu hors-écran dans un buffer ayant une taille de 512x512 mais que la fenêtre -et donc les buffers qui lui sont liés- a elle une taille de 1280x720.

Dans ce cas, il est important de re-spécifier les dimensions du viewport afin que les opérations de fenêtre de OpenGL agissent sur les bonnes dimensions.

Exemple 2:

```
int windowWidth = 1280, windowHeight = 720;
int offscreenWidth = 512, offscreenHeight = 512;

glBindFramebuffer(GL_FRAMEBUFFER, FBO);          // rendu hors écran
glViewport(0, 0, offscreenWidth, offscreenHeight);
glClearColor(1.f, 0.f, 0.f, 1.f);                // efface l'écran en rouge pour
test
glClear(GL_COLOR_BUFFER_BIT);                    // juste un color-buffer
...
glBindFramebuffer(GL_FRAMEBUFFER, 0); // ne pas oublier de rétablir la bonne taille
de viewport
glViewport(0, 0, windowWidth, windowHeight);
glClearColor(1.f, 1.f, 1.f, 1.f);                // Efface le back-buffer en
blanc
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // efface le color-buffer et
depth-buffer
```

3. Attacher des buffers

Le type et nombres de buffers que nous allons devoir attacher au FBO dépend de l'usage. Pour une application standard il nous faut au moins un color-buffer pour stocker les couleurs des pixels. Dans le cadre d'une application 3D il nous faut également un depth-buffer.

Comme nous l'avons vu, un buffer pour nous est essentiellement une texture.

On peut donc reprendre le code de notre fonction LoadTexture() et ne gardant que les fonctions qui ont trait à la création et paramétrage de la texture.

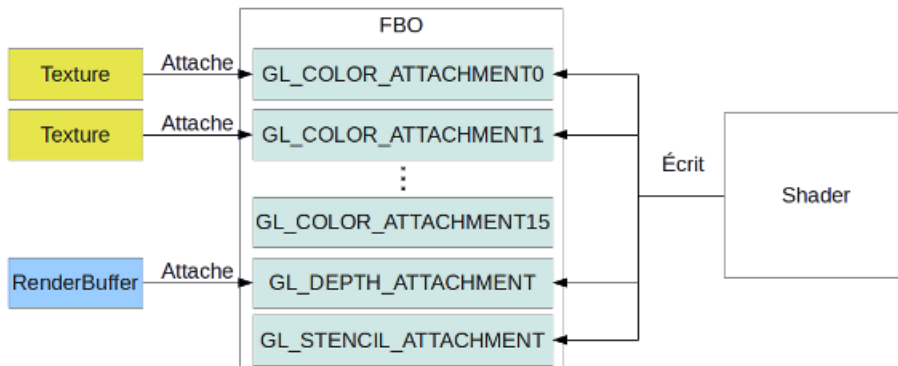
Les "attachments" (points d'attache) d'une texture sur un FBO sont nombreux, les deux plus fondamentaux sont:

- GL_COLOR_ATTACHMENT0 – le premier (et par défaut) point d'attache d'un color-buffer
Comme son nom l'indique, il est en théorie possible d'attacher plusieurs color-buffers.
- GL_DEPTH_ATTACHMENT – le seul point d'attache d'un depth-buffer
- GL_STENCIL_ATTACHMENT – le seul point d'attache d'un stencil-buffer (on en reparlera)

Pour attacher explicitement un buffer (texture ici) à un Framebuffer (FBO) on utilise la fonction **glFramebufferTexture2D()** que l'on peut comprendre comme "j'attache une Texture2D à un Framebuffer". Les paramètres sont

glFramebufferTexture2D(GLenum target, GLenum attachment, GLenum textureTarget, GLuint textureID, GLint mipmapLevel)

- Target – GL_FRAMEBUFFER, GL_READ_FRAMEBUFFER ou GL_DRAW_FRAMEBUFFER
- Attachment – GL_COLOR_ATTACHMENT0 ou GL_DEPTH_ATTACHMENT pour ce TP
- TextureTarget - TEXTURE_2D pour ce TP
- TextureID – L'identifiant de texture créé par glGenTextures()
- MipmapLevel – 0 (zéro) pour ce TP, mais il est possible de modifier n'importe quel LOD



Exemple 3:

```
// après avoir créer les textures des color-buffers, depth-buffer etc..
// note: il n'est pas nécessaire de "bind" les textures afin de les attacher à un FBO
glBindFramebuffer(GL_FRAMEBUFFER, FBO);
...
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
                       GL_TEXTURE_2D, fb.colorBuffer, 0);
```

4. Créer un depth-buffer (texture)

Dans notre application, un depth-buffer est avant tout une texture mais avec un format de pixel particulier.

En l'occurrence, les valeurs stockées dans un depth-buffer sont soit des int (GL_UNSIGNED_INT) soit des float (GL_FLOAT). Le depth-buffer stocke une seule composante dans son format qui est appelée GL_DEPTH_COMPONENT.

En interne il faut spécifier la profondeur (en bits) du depth-buffer. Cela peut être 16, 24 ou 32 bits.

Exemple de création d'un depth-buffer entier de 24 bits

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT24, w, h, 0, GL_DEPTH_COMPONENT,
             GL_UNSIGNED_INT, NULL);
```

Exemple de création d'un depth-buffer réel de 32 bits (notez le 32F)

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT32F, w, h, 0, GL_DEPTH_COMPONENT,
             GL_FLOAT, NULL);
```

Reste à l'attacher en utilisant cette fois-ci `GL_DEPTH_ATTACHMENT` comme valeur d'attachement.

5. Vérifier les erreurs

Il est temps d'aborder un aspect qui va devenir très important qui est celui du débogage.

Les versions d'OpenGL offrent une couche d'information sur les erreurs qui est plutôt spartiate.

La fonction `glGetError()` renvoie un `GLenum` spécifiant le dernier code d'erreur retourné par OpenGL.

Cela signifie que l'on ne peut pas savoir précisément quelle fonction est à l'origine de l'erreur, et de plus, le code d'erreur a une signification différente pour chacune des fonctions de l'API OpenGL.

Il est recommandé d'ajouter un appel à `glGetError()` à chaque appel de fonction afin d'intercepter au plus tôt une éventuelle erreur.

Cependant, comme les appels de type `glGet***()` sont assez coûteux il vaut mieux n'utiliser ces fonctions que durant la phase de développement et surtout pas pour la version finale (release) hormis les cas extrêmes.

Vous pouvez au choix définir une fonction spéciale pour ce type de vérification ou bien utiliser une "assertion" du C++ qui va forcer votre programme à s'arrêter dans le debugger dès que surgit une valeur inattendue. Exemple :

```
#include <assert.h> // ou <cassert>

// l'assertion bloque l'exécution dès que la valeur de retour de glGetError() est
différente de NO_ERROR
assert(glGetError() == GL_NO_ERROR);
```

Dans le cas des Framebuffers en particulier nous disposons de la fonction `glCheckFramebufferStatus()`.

Cette fonction renvoie un code d'erreur spécifique indiquant un problème dans la validation du FBO.

La valeur **`GL_FRAMEBUFFER_COMPLETE`** indique que tous les paramètres sont ok et que le FBO est utilisable pour le rendu.

```
#include <assert.h> // ou <cassert>

// alternativement vous pouvez gérer tous les différents cas avec un switch / case
assert(glCheckFramebufferStatus(GL_FRAMEBUFFER) == GL_FRAMEBUFFER_COMPLETE);
```

6. Exercices

Exercice 1.1

- a. créez une texture pour le color-buffer ainsi qu'une texture pour le depth-buffer. Ils doivent tous deux avoir les mêmes dimensions que votre fenêtre.
- b. créez un FBO avec ces deux textures
- c. effectuez le rendu de votre application dans le FBO
- d. copiez le contenu du COLOR_ATTACHMENT0 de votre FBO dans le back buffer à l'aide de la fonction `glBlitFramebuffer()` <https://www.khronos.org/opengl/wiki/GLAPI/glBlitFramebuffer>

Avant d'appeler `glBlitFramebuffer()` il faut indiquer à OpenGL quelles sont les sources et destinations. On utilise simplement `glBindFramebuffer()` en indiquant la source en `GL_READ_FRAMEBUFFER` et la destination en `GL_DRAW_FRAMEBUFFER`.

Exercice 1.2

Plutôt que d'utiliser la fonction `glBlitFramebuffer()` il est possible de customiser la copie en utilisant un rendu plein écran ainsi qu'un shader assez simple.

- e. créez un VBO+VAO afin d'afficher un quadrilatère plein écran (-1;-1) pour les 4 coins en NDC.
- f. créez et compilez un shader (vertex + fragment) qui sample une texture et écrit la couleur du texel dans `gl_FragColor`
- g. A la place de l'étape d de l'exercice 1.1 affichez votre triangle (strip) en faisant attention à bien configurer les états d'OpenGL.

En pratique, pour une application 3D, il est usuel de séparer la boucle de rendu en au moins deux parties. D'une part le code 3D, puis le code de post-traitement (2D).

Dans le cas 3D, on a coutume d'activer le face-culling ainsi que le test et l'écriture dans le depth-buffer.

```
// 3D
glEnable(GL_CULL_FACE);
glEnable(GL_DEPTH_TEST);
glDepthMask(GL_TRUE);      // attention, le DepthMask doit être actif AVANT d'appeler
glClear()
```

Tandis qu'en 2D, on fait exactement le contraire.

```
glDisable(GL_CULL_FACE);
glDisable(GL_DEPTH_TEST); // Desactive le test (lecture) avec le depth-buffer
glDepthMask(GL_FALSE);   // Desactive l'écriture dans le depth-buffer
```

```
// code d'exemple pour rendre votre quadrilatere
GLuint program = copyShader.getProgram();
glUseProgram(program);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, FBO.color_buffer);
glBindVertexArrays(copyVAO);
```

```
glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);  
...
```

Partie 2 – Applications

Les applications sont nombreuses. Dans sa forme primaire on peut utiliser un FBO pour générer une texture de manière procédurale, appliquer un effet de post-traitement en une passe ou encore stocker des paramètres de rendu par pixel.

Nous allons voir comment appliquer deux de ces techniques à notre programme.

Il est recommandé de séparer la fonction d'initialisation (et terminaison) en 3 parties, une pour le rendu standard, une pour la génération de texture procédurale et une autre partie pour le post-traitement (similaire à l'exercice 1.2).

Exercice 2.1: Post-traitement

Sur la base de l'exercice 1.2, modifiez le fragment shader afin d'effectuer un post-traitement à la volée (c'est à dire sans recopie, en samplant la texture et en modifiant la couleur en sortie).

Voici quelques exemples de post-traitement simple en une passe:

Luminance (grayscale)

On pourrait croire que l'intensité lumineuse (appelée luminance) est une simple moyenne des composantes r, g et b d'un pixel.

Cependant, comme l'intensité est perceptuelle, il faut pondérer la contribution des rouges, verts et bleus en fonction de la sensibilité de la vision humaine. Ces poids sont de (0.2125, 0.7154, 0.072). La luminance étant une valeur réelle (un scalaire), ce que l'on souhaite faire est une somme de produits entre les poids et chaque composantes, cela correspond complètement à la définition du produit scalaire:

```
const vec3 luminanceWeight = vec3 (0.2125, 0.7154, 0.072);  
float luminance = dot(color.rgb, luminanceWeight);
```

Sépia <https://fr.wikipedia.org/wiki/S%C3%A9pia>

Même principe que pour la luminance sauf que cette fois-ci on calcule 3 produits scalaires avec 3 groupes de poids différents pour R, G et B.

```
vec3 sepia = vec3(  
    dot(color.rgb, vec3(0.393, 0.769, 0.189)),  
    dot(color.rgb, vec3(0.349, 0.686, 0.168)),  
    dot(color.rgb, vec3(0.272, 0.534, 0.131))  
);
```

Noir & Blanc

Plutôt “noir ou blanc”, il s’agit ici de n’afficher qu’un rendu binaire, un peu comme dans Renaissance, Onyx Film.

On fait la moyenne des valeurs R, G et B. Si la moyenne est supérieure à 0.5 on affiche du blanc sinon du noir.

Autre exercice intéressant: conversion RGB->HSV (ou HSL ou HSB) et manipulation de la teinte (H = Hue) ou de la saturation (S) des pixels.

Vous pouvez par exemple passer en uniform la nouvelle valeur de teinte et/ou saturation et ainsi recoloriser l’image de manière plus réaliste qu’avec une simple multiplication.

Exercice 2.2: Post-traitement avec noyau de convolution (kernel)

Il s’agit ici d’effectuer une forme limitée de convolution, à savoir le fait de prendre en compte l’ensemble des données dans un rayon défini pour le calcul d’un élément particulier.

Dans cette technique, le calcul d’un pixel nécessite de prendre en compte un certain nombre de pixels adjacents. Dans la forme la plus basique, on peut représenter ce noyau (kernel) sous la forme d’une matrice de pixels centrée autour du pixel courant.

Le contenu de la matrice est interprété comme une pondération, c’est-à-dire que l’on va effectuer une somme pondérée des pixels adjacents.

La première question soulevée est : comment récupérer la position des pixels voisins dans une texture alors que les coordonnées de texture sont normalisés [0,+1] ?

Tout simplement, il suffit de passer les dimensions de la texture échantillonnée et/ou la taille d’un pixel de la dite texture en tant que variable uniforme.

Lire 1 pixel à gauche implique d’ajouter un offset de $(-1.0/\text{imageWidth}, 0.0)$, lire un pixel immédiatement en bas implique un offset de $(0.0, 1.0/\text{imageHeight})$, et un pixel en bas à gauche implique un offset de $(-1.0/\text{imageWidth}, +1.0/\text{imageHeight})$ etc...

Références

http://www.songho.ca/opengl/gl_fbo.html

<http://www.opengl-tutorial.org/fr/intermediate-tutorials/tutorial-14-render-to-texture/>

<https://alexandre-laurent.developpez.com/tutoriels/OpenGL/OpenGL-FBO/>

<https://learnopengl.com/Advanced-OpenGL/Framebuffers>